

Selecting a column:

name	age	city	Salary
John	25	chennai	35000
david	30	mumbai	45000
Sara	28	Delhi	40000

Dataset

Select one column:

```
print(dt["name"])
```

Output:

0 John
1 David
2 Sara

name: name dtype: object

> string consider as object.

Select only one column is $\Rightarrow$ series

Selecting Multiple columns:

Syntax:

```
dt(["column1", "column2"])
```

Single bracket $\rightarrow$ Returns the Series

Double bracket $\rightarrow$ Returns the Datframe.

Selecting Rows Using loc:

Syntax:

```
dt.loc [row, column]
```

it select Rows & column
Using labels (names or index)

`print(dt.loc [0])` $\rightarrow$ it returns the first row

Output: `[1])` $\rightarrow$ it returns 2nd row

Multiple rows:

```
print(dt.loc [[0, 2]])
```

out: John row &
Sara row

Selecting Specific rows and columns:

```
print(dt.loc [:, ["name", "salary"]])
```

$\Rightarrow$ Means all rows.

Selecting rows Using `iloc()`

* Selecting data Using Integer position

`iloc` $\Rightarrow$ `i` = Integer

Interview qn:

`loc`:

Uses labels

`dt.loc[0]`

Column names

`iloc`:

Uses positions

`dt.iloc[0]`

column numbers

Manager ask employee whose salary > 40000

Use Filtering Concept

Filtering Data:

`print(dt[dt["salary"] > 40000])`

ex:

qn: age > 25

`print(dt[dt["age"] > 25])`

qn: where city chennai:

`print(dt[dt["city"] == "chennai"])`

Multiple Condition:

Employee From chennai, salary above 40000

`print(dt[(dt["city"] == "chennai") & (dt["salary"] > 40000)])`

AND - &

OR - ||

First call dt, then apply dt to condition.

Sorting By INDEX:

Syntax:

df.sort_index()

* Sorting the Rows based on the index.

Adding a New Column:

df["department"] = ["HR", "IT", "Finance"]

Column Rename:

df.rename(columns = {"name": "employee
name"})

Permanent Save:

inplace = True)

Deleting a column:

df.drop("bonus", axis=1)

axis 1 means column:

Multiple column delete:

df.drop(["bonus", "department"], axis=1)

Deleting rows:

df.drop(0)

It deleted the first row.

Qn:

1. Display only the name column
2. Display name & Salary columns
3. Display students whose marks are greater than 80
4. Display employees from Chennai
5. Sort the dataset based on Salary in descending order?
6. Add a bonus column equal to 15% of Salary?

7. Rename the
employee name?

8. Delete the bonus column?

Data cleaning:

* It's the process of identifying and correcting incorrect, incomplete, duplicate or missing data

NAN $\Rightarrow$ not available. (means data missing)
 $\Rightarrow$ not a number

Missing Values:

A information is Unavailable.

`IsNull()` $\rightarrow$ Finding the missing values.

```
import pandas as pd
dt = pd.read_csv("student.csv")
print(dt.isnull())
```

Output:

name	false	$\rightarrow$ no missing value
age	true	$\rightarrow$ value missing
city	false	
Salary	true	

Counting Missing Values:

```
Print (dt.isnull().sum())
```

output:

name	0
age	2
city	0
Salary	1

Remove Missing values:

```
dropna()
```

$\Rightarrow$ `dt.dropna()`

Rows Containing Missing

values are Removed.

7. Rename the column name to employee_name?

8. Delete the bonus column?

Data Cleaning:

* It's the process of identifying and correcting incorrect, incomplete, duplicate or missing data

NAN $\Rightarrow$ not available. (means data missing)
 $\Rightarrow$ not a number

Missing Values:

A information is Unavailable.

`isnull()` $\rightarrow$ Finding the missing values.

```
import pandas as pd
```

```
dt = pd.read_csv("student.csv")
```

```
print(dt.isnull())
```

Output:

name	false	$\rightarrow$ no missing value
age	true	$\rightarrow$ value missing
city	false	
Salary	true	

Counting Missing Values:

```
Print (dt.isnull().sum())
```

output:

name	0
age	2
city	0
Salary	1

Remove Missing values:

```
dropna()
```

```
 $\Rightarrow$  dt.dropna()
```

Rows Containing Missing values are Removed.

dt.dropna(inplace=True) $\Rightarrow$ Permanently delete

Fill the Missing Values:

fillna()

[missing values 0 filled]

Replace with zero

dt.fillna(0)

Replace with average

dt["age"].fillna(dt["age"].mean(),
inplace=True)

dt["age"] = dt["age"].
fillna(dt["age"].mean())

20
25
30
Nan } $\rightarrow$ Mean 25
 $\rightarrow$ changed to 25

20
25
30
25

Replace with Median()

dt["age"].fillna(dt["age"].median(),
inplace=True)

Replace with mode(): $\Rightarrow$ Most Frequently occurring value

* Mostly Categorical data.

dt["city"].fillna(dt["city"].mode(), inplace=True)

Datatype Best choice

Numerical $\rightarrow$ Mean

Skewed numerical $\rightarrow$ Median

Categorical data $\rightarrow$ Mode

[Interview qn]

Duplicate Value check:

print(dt.duplicated())

Output:

False

False

True $\leftrightarrow$ Duplicate

Remove duplicate:

```
dt.drop_duplicates()
```

Permanently Remove duplicate:

```
dt.drop_duplicates (inplace=True)
```

Qn:

1. Find Missing Values in the dataset?
2. Count the number of missing values in each column?
3. Replace missing age values with average age?
4. Remove duplicate Rows?